Introduction to Strings

A **string** is a sequence of characters enclosed within single quotes (`' '`), double quotes (`" "`), or triple quotes (`''' '''` or `""" """`).
Strings are **immutable**, meaning once created, they cannot be changed.

Example:

```python
# Defining strings
str1 = 'Hello'
str2 = "World"
str3 = '''This is a multi-line
string example.'''

print(str1)
print(str2)
print(str3)
```

Output:

```
Hello
World
This is a multi-line
string example.
```

Operations on Strings

1. **Concatenation (`+`)** - Joining two or more strings.
2. **Repetition (`*`)** - Repeating a string multiple times.
3. **Indexing** - Accessing individual characters using indices.
4. **Slicing** - Extracting a part (substring) of the string.
5. **Membership (`in`, `not in`)** - Checking if a substring exists within a string.

Examples:

```python
# Concatenation
str1 = "Hello"
str2 = "World"
result = str1 + " " + str2
print(result)  # Output: Hello World

# Repetition
print("Python! " * 3)  # Output: Python! Python! Python!

# Indexing
text = "Python"
print(text[0])    # Output: P (1st character)
print(text[-1])   # Output: n (last character)

# Slicing
print(text[0:3])  # Output: Pyt (from index 0 to 2)
print(text[:4])   # Output: Pyth (default start is 0)
print(text[2:])   # Output: thon (from index 2 to end)

# Membership
print('Pyt' in text)    # Output: True
print('Java' not in text)  # Output: True
```

Built-in String Methods

String methods are functions built into Python to perform operations on strings.

Common Methods:

1. **`upper()`** - Converts all characters to uppercase.
2. **`lower()`** - Converts all characters to lowercase.
3. **`strip()`** - Removes leading and trailing whitespaces.
4. **`replace(old, new)`** - Replaces a substring with another.
5. **`split(separator)`** - Splits the string into a list based on a separator.
6. **`join(iterable)`** - Joins elements of an iterable with a separator.
7. **`find(sub)`** - Returns the index of the first occurrence of the substring.
8. **`count(sub)`** - Counts the occurrences of a substring.

9. **`startswith(sub)` / `endswith(sub)`** - Checks if a string starts/ends with a specific substring.
10. **`isalpha()` / `isdigit()` / `isspace()`** - Checks for alphabetic, digit, or whitespace characters.

Examples:

```python
# 1. upper() and lower()
text = "Hello World"
print(text.upper())  # Output: HELLO WORLD
print(text.lower())  # Output: hello world

# 2. strip()
text = "   Python   "
print(text.strip())  # Output: Python

# 3. replace()
print(text.replace("Python", "Java"))  # Output:    Java

# 4. split() and join()
sentence = "apple,banana,grape"
fruits = sentence.split(",")
print(fruits)  # Output: ['apple', 'banana', 'grape']

joined = "-".join(fruits)
print(joined)  # Output: apple-banana-grape

# 5. find() and count()
quote = "Life is beautiful because life is simple."
print(quote.find("life"))    # Output: -1 (case-sensitive)
print(quote.lower().find("life"))  # Output: 27 (after
converting to lowercase)
print(quote.count("is"))     # Output: 2

# 6. startswith() / endswith()
filename = "document.pdf"
print(filename.startswith("doc"))  # Output: True
print(filename.endswith(".pdf"))   # Output: True

# 7. isalpha(), isdigit(), isspace()
```

```
print("Python".isalpha())  # Output: True
print("12345".isdigit())   # Output: True
print("    ".isspace())     # Output: True
```

Built-in String Functions

While string **methods** are called on string objects, Python also provides some **built-in functions** that work with strings:

1. `len()` - Returns the length of the string.
2. `max()` / `min()` - Returns the maximum or minimum character (based on ASCII values).
3. `sorted()` - Returns a sorted list of characters.
4. `chr()` / `ord()` - Converts between characters and their ASCII codes.
5. `format()` - Formats strings dynamically.

Examples:

Python

```
# 1. len()
text = "Hello World"
print(len(text))  # Output: 11

# 2. max() and min()
print(max("abcXYZ"))  # Output: 'c' (highest ASCII value)
print(min("abcXYZ"))  # Output: 'X' (lowest ASCII value)

# 3. sorted()
print(sorted("python"))  # Output: ['h', 'n', 'o', 'p', 't',
'y']

# 4. chr() and ord()
print(ord('A'))  # Output: 65 (ASCII value of 'A')
print(chr(97))   # Output: 'a' (character for ASCII value 97)

# 5. format()
name = "Alice"
age = 25
```

```
print("My name is {} and I am {} years old.".format(name,
age))
# Output: My name is Alice and I am 25 years old.

# Using f-strings (Python 3.6+)
print(f"My name is {name} and I am {age} years old.")
# Output: My name is Alice and I am 25 years old.
```

Complete List of Python String Methods with Examples

Python provides many built-in string methods to perform various operations. Here's the **complete list** of string methods along with **examples** and their **outputs**.

1. Case Conversion Methods

Method	Description	Example
`upper()`	Converts all characters to **uppercase**.	`"hello".upper()` → `"HELLO"`
`lower()`	Converts all characters to **lowercase**.	`"HELLO".lower()` → `"hello"`
`capitalize()`	Capitalizes the **first character**.	`"python".capitalize()` → `"Python"`
`title()`	Capitalizes the **first letter of each word**.	`"hello world".title()` → `"Hello World"`
`swapcase()`	**Swaps case**: upper → lower, lower → upper.	`"PyThOn".swapcase()` → `"pYtHoN"`
`casefold()`	Converts to lowercase (more aggressive than `lower()`).	`"ß".casefold()` → `"ss"`

2. Alignment & Formatting Methods

Method	Description	Example
`center(width, fillchar)`	Centers the string within `width`.	`"python".center(10, "*")` → `"*python*"`
`ljust(width, fillchar)`	Left-aligns the string within `width`.	`"py".ljust(5, "-")` → `"py---"`
`rjust(width, fillchar)`	Right-aligns the string within `width`.	`"py".rjust(5, "-")` → `"---py"`
`zfill(width)`	Pads the string with **zeros** on the left.	`"42".zfill(5)` → `"00042"`
`format()`	Formats strings dynamically.	`"Name: {}, Age: {}".format("Tom", 25)` → `"Name: Tom, Age: 25"`
`format_map(mapping)`	Formats using a dictionary.	`"{name} is {age}".format_map({'name': 'Tom', 'age': 25})` → `"Tom is 25"`

3. Search & Find Methods

Method	Description	Example
`find(sub)`	Returns the **index** of first occurrence, `-1` if not found.	`"hello".find("l")` → `2`
`rfind(sub)`	Returns the **last occurrence** of the substring.	`"hello".rfind("l")` → `3`
`index(sub)`	Like `find()`, but raises an **error** if not found.	`"hello".index("e")` → `1`
`rindex(sub)`	Like `rfind()`, but raises an **error** if not found.	`"hello".rindex("l")` → `3`
`count(sub)`	Counts how many times `sub` appears.	`"banana".count("a")` → `3`

4. String Testing Methods (Boolean Results)

Method	Description	Example
`startswith(sub)`	Checks if the string starts with `sub`.	`"python".startswith("py")` → `True`
`endswith(sub)`	Checks if the string ends with `sub`.	`"python".endswith("on")` → `True`
`isalpha()`	Returns `True` if all characters are **alphabets**.	`"Hello".isalpha()` → `True`
`isdigit()`	Returns `True` if all characters are **digits**.	`"1234".isdigit()` → `True`
`isalnum()`	Returns `True` if all characters are **alphanumeric**.	`"abc123".isalnum()` → `True`
`islower()`	Returns `True` if all characters are **lowercase**.	`"hello".islower()` → `True`
`isupper()`	Returns `True` if all characters are **uppercase**.	`"HELLO".isupper()` → `True`
`isspace()`	Returns `True` if all characters are **whitespace**.	`" ".isspace()` → `True`
`istitle()`	Returns `True` if the string is in the title **case**.	`"Hello World".istitle()` → `True`
`isdecimal()`	Returns `True` if all characters are **decimal numbers**.	`"123".isdecimal()` → `True`
`isnumeric()`	Returns `True` if all characters are **numeric**.	`"⅛".isnumeric()` → `True`
`isidentifier()`	Checks if the string is a valid **Python identifier**.	`"variable1".isidentifier()` → `True`

5. Replace & Modify Methods

Method	Description	Example
`replace(old, new)`	Replaces occurrences of `old` with `new`.	`"apple".replace("p", "b")` → `"abble"`
`translate(table)`	Replaces characters using a **translation table**.	`"abc".translate(str.maketrans("a", "x"))` → `"xbc"`
`maketrans()`	Creates a translation table for `translate()`.	`str.maketrans("ae", "12")`

6. Splitting & Joining Methods

Method	Description	Example
`split(sep)`	Splits the string into a list by `sep`.	`"a,b,c".split(",")` → `['a', 'b', 'c']`
`rsplit(sep)`	Splits from the **right** side.	`"a,b,c".rsplit(",", 1)` → `['a,b', 'c']`
`splitlines()`	Splits at **line breaks** (`\n`).	`"Hello\nWorld".splitlines()` → `['Hello', 'World']`
`join(iterable)`	Joins elements with a **separator**.	`" ".join(["Hello", "World"])` → `"Hello World"`
`partition(sep)`	Splits into a **3-part tuple** at first `sep`.	`"apple-pie".partition("-")` → `('apple', '-', 'pie')`
`rpartition(sep)`	Splits into a **3-part tuple** at last `sep`.	`"apple-pie".rpartition("-")` → `('apple', '-', 'pie')`

7. Whitespace & Trimming Methods

Method	Description	Example
`strip(chars)`	Removes **leading and trailing** characters (default: spaces).	`" hello ".strip()` → `"hello"`
`lstrip(chars)`	Removes **leading** characters.	`"---hello".lstrip("-")` → `"hello"`
`rstrip(chars)`	Removes **trailing** characters.	`"hello---".rstrip("-")` → `"hello"`

8. Encoding & Decoding Methods

Method	Description	Example
`encode(encoding)`	Converts the string to **bytes**.	`"hello".encode("utf-8")` → `b'hello'`
`decode(encoding)`	Converts bytes back to **string**.	`b'hello'.decode("utf-8")` → `"hello"`